

WEEK 7 · LECTURE 13

Building a Support Copilot

Part 1: the layer that reads

5,000 tickets a week, three agents, and not one labelled example. Let's ship something anyway.

WEEK 7 · AT A GLANCE

Language Models & Transformers

What we cover across both lectures, the models we actually run, and what to read alongside them.

CONCEPTS · WEEK 7

Generalised language models

one pretrained model, pointed at many tasks

L13

Tokenization & embeddings

sub-words in, vectors out

L13

Transfer without training

zero-shot classification as entailment

L13

Hugging Face Transformers

pipeline, from_pretrained, model cards

L13

BERT & masked language modelling

how the reading ability was learned

L14

Encoder vs decoder

why BERT reads and GPT writes

L14

Pretraining → fine-tuning

what a few hundred labels buy you

L14

RoBERTa & training from scratch

same architecture, better recipe

L14

MODELS WE ACTUALLY RUN

MODEL	SIZE	TYPE	USED FOR
all-MiniLM-L6-v2 sentence-transformers	23M · 6 layers · 384-dim	encoder	embeddings & search L13
deberta-v3-base-zeroshot MoritzLaurer	184M · 12 layers · 768-dim	encoder	zero-shot routing L13
bert-base-uncased Google	109M · 12 layers · 768-dim	encoder	masked LM demo L14
distilbert-base-uncased Hugging Face	66M · 6 layers · 768-dim	encoder	fine-tuning L14
gpt2 OpenAI	124M · 12 layers · 768-dim	decoder	the contrast case L14

Every one of these is **pretrained by someone else** and downloaded for free. Nothing this week is trained from scratch. That is the whole point.

READINGS

J&M Ch. 7 [Large Language Models](#)

J&M Ch. 10 [BERT & masked LMs](#)

HOLLM Ch. 3 [inside transformer LLMs](#)

Tunstall Ch. 10 [training from scratch](#)

Vaswani 2017 [Attention Is All You Need](#)

Devlin 2018 [BERT](#)

Liu 2019 [RoBERTa](#)

THE BUSINESS

First, what company are we walking into?

Before any technology, you need to be able to picture the job. Here it is in three sentences.

1 · What Cobalt sells

Software, sold by subscription to about 600 other businesses. It plugs into their databases and turns the numbers into dashboards their staff check every morning.

2 · Why customers write in

When something breaks for them (a chart shows a wrong number, nobody can sign in, an invoice looks odd) they click **Help** → **Contact support**, or email support@cobalt.com, and describe it in their own words.

3 · Where it lands

Each message becomes a **ticket**, dropped into one shared queue the three agents work through, oldest first. **That queue is what we will call the inbox.**

TWO REAL TICKETS FROM THE QUEUE

CB-4207 · in-app · Business plan · EMEA

"Dashboards take 40+ seconds to load since yesterday. They used to come up instantly."

CB-4288 · email · Team plan · North America

"We were charged twice this month for the same seats. We need a refund on the duplicate."

A ticket is **a few sentences of ordinary writing**, plus who sent it. No form, no dropdown, no category.

Five thousand of these arrive every week. Three people answer them.

THE PROBLEM

Every ticket has to be read before it can be answered.

And that reading is where the three agents lose their day.

~5,000 tickets a week

and three people to answer them

~40% are repeats

the same handful of questions

Nothing is sorted

no tags, no categories, just text

Minutes lost per ticket

just working out who should own it

WHY THAT THIRD BOX IS THE HARD ONE

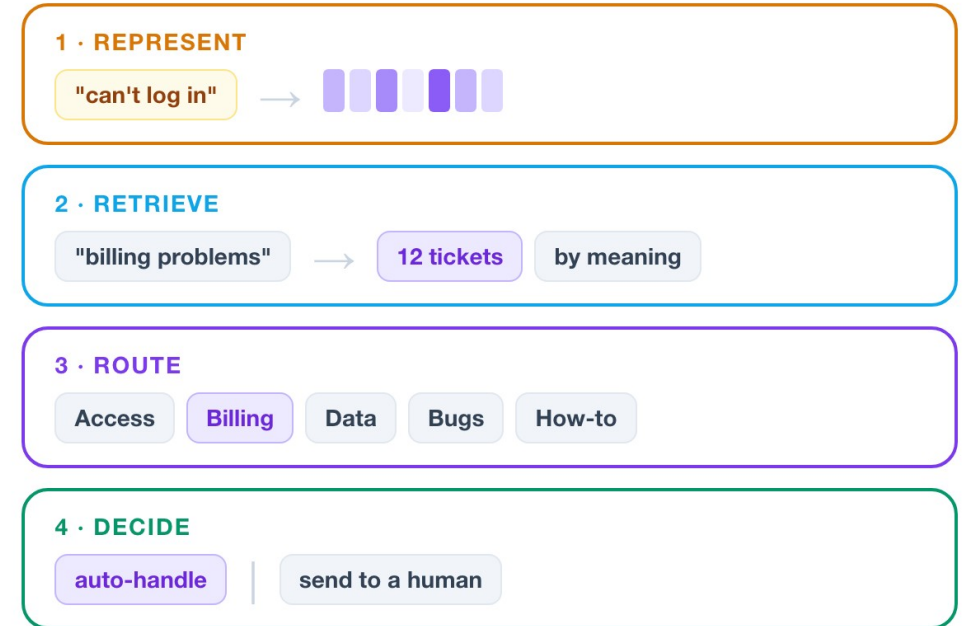
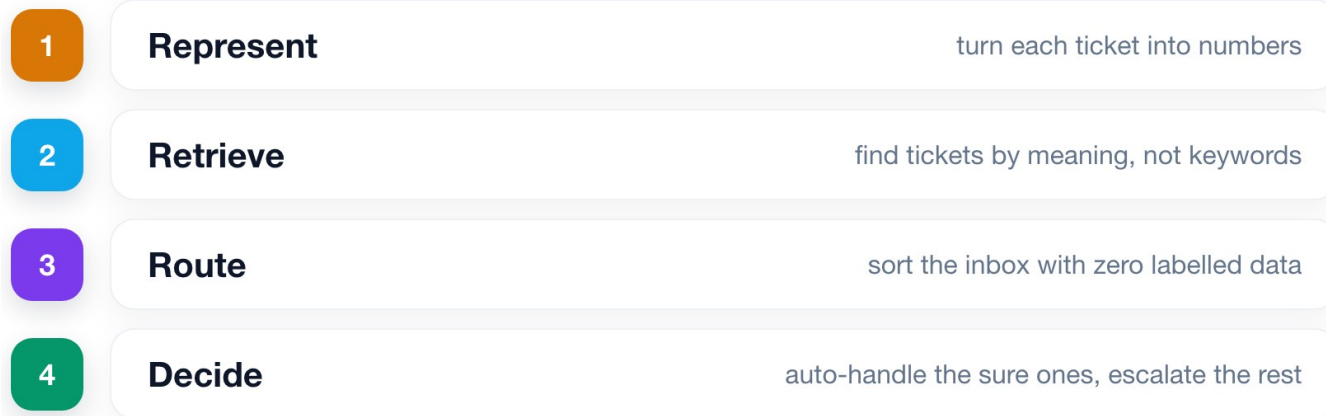
The normal way to teach a computer to sort text is to hand it a few thousand examples that a person has already sorted by hand. Those hand-sorted examples are called training data, and most machine learning cannot start without them. Cobalt has none. Building that pile would mean paying people to read and tag thousands of tickets before you could write a line of code.

So tonight we build something that never needs them. We start with the half that READS the inbox: search it, sort it, and decide what a person actually has to see.

THE PLAN

Four capabilities, built in order.

Each one is useful on its own. Together they are a product.



A model cannot read. It can only do arithmetic.

THE CODE

```
enc = SentenceTransformer(
    "all-MiniLM-L6-v2")

tok = enc.tokenizer
tok.tokenize(
    "Postgres connector failed")
```

THE MODEL NEVER SEES WORDS

our

post

##gre

##s

connector

failed

.

A fixed vocabulary of ~30,000 pieces. Anything rare gets **spelled out** from fragments, which is how it copes with a word it has never seen.

Check what happens to your business vocabulary.

THE CODE

```
for w in ["Okta", "SAML", "webhook",  
"Snowflake", "Kubernetes"]:  
    print(w, "->", tok.tokenize(w))
```

YOUR PRODUCT VOCABULARY

Okta	-> ok + ##ta
SAML	-> sam + ##l
webhook	-> web + ##ho + ##ok
Snowflake	-> snow + ##fl + ##ake
Kubernetes	-> ku + ##ber + ##net + ##es

The model has never seen your industry's words. It is sounding them out phonetically. This is why semantic search degrades exactly where your business is most specific.

One ticket becomes one vector, by averaging.

WHAT ENCODE() ACTUALLY DOES

```
v = bert(**batch).last_hidden_state
# one vector PER TOKEN

m = batch["attention_mask"]
pooled = (v * m).sum(1) / m.sum(1)
pooled = normalize(pooled)
```

WHAT YOU GET BACK

```
per token (1, 12, 384)
           |
           v average them
per ticket (1, 384)

matches encoder.encode() exactly:
max difference = 0.0
```

One ticket = **the average of its token vectors**. Which is also the weakness: a long ticket covering three problems blends into one mushy point. That is why long documents get chunked.

What actually happened inside bert(...)?

You just called a transformer and got vectors back. Ninety seconds on what it did with them.

ONE TRANSFORMER BLOCK

1 • Attention: look around

Every word looks at every other word and pulls in what it needs. Information moves **between** words.



2 • Feed-forward: think about it

Each word then goes through a small network **on its own**, which applies what the model knows about it. No looking sideways.



then do the whole thing again, 12 times

Attention moves information

After this step, the vector for "**charged**" carries a piece of "twice" and "seats" inside it. That is how a word comes to mean what it means *in this ticket*.

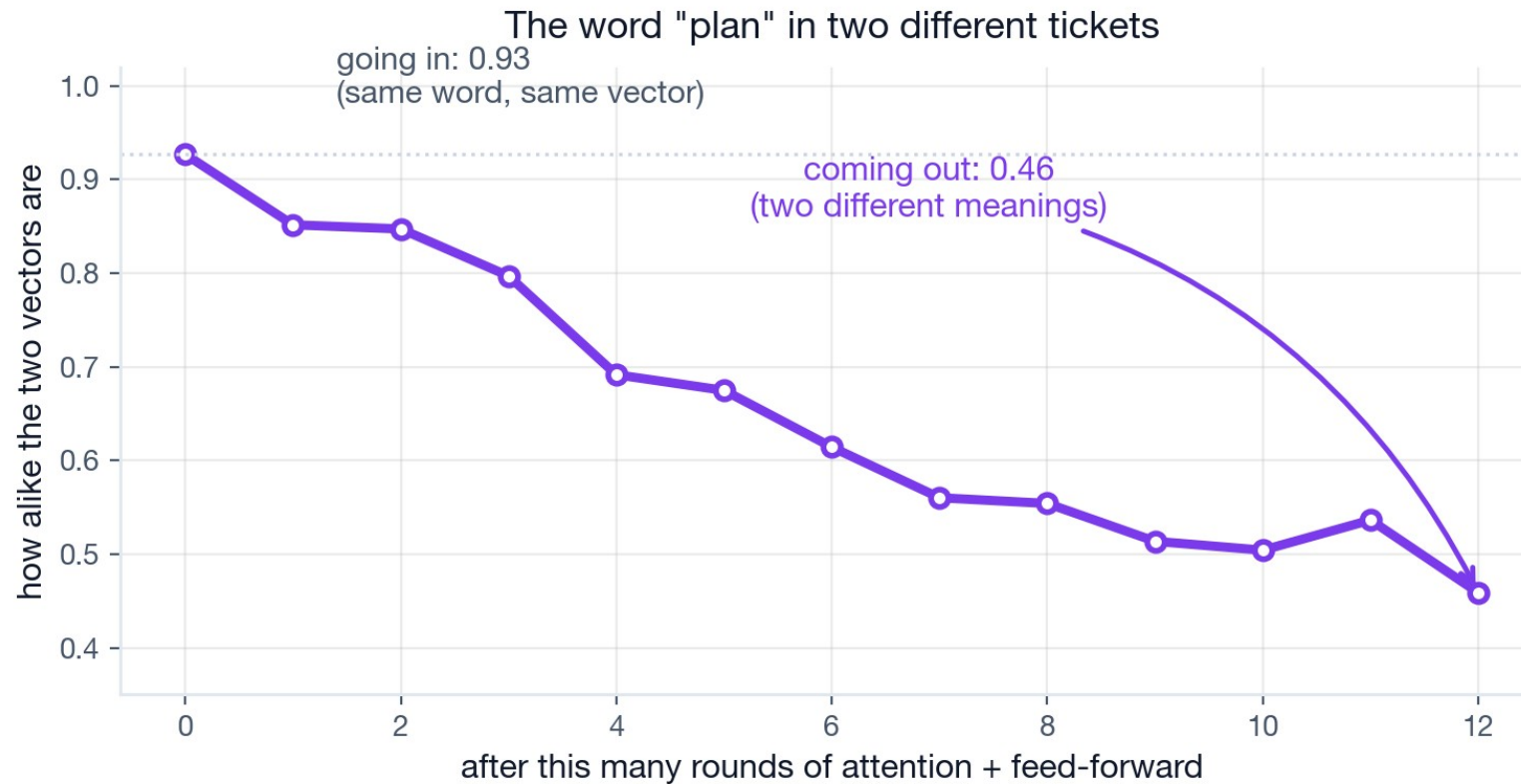
Feed-forward applies knowledge

This is where facts live. "Billed every ____" → "month". Nothing is mixed between words here; each one is processed alone.

That pair is the entire machine. **Look around, then think about it.** Stack it twelve times and you have the model we have been calling all night.

A word absorbs the words around it.

"Plan" is a subscription tier in one ticket and an intention in another. Same word, same starting vector.



Going in they are 0.93 alike. Twelve rounds later, 0.46. That gap is attention doing its job.

Where the model keeps what it knows.

ASK IT SOMETHING IT WAS NEVER TOLD

```
from transformers import pipeline
fill = pipeline("fill-mask",
model="bert-base-uncased")

fill("Our subscription is billed
every [MASK].")
```

IT KNOWS HOW SUBSCRIPTIONS WORK

```
month    0.69
year     0.09
week     0.08

# nobody programmed this in.
# it was learned, and it is stored
# in the feed-forward weights.
```

attention

33%

feed-forward

67%

Two thirds of every layer is feed-forward, only a third is attention. Attention gets the headlines; the knowledge lives next door.

Similarity is a multiplication.

THE CODE

```
a = encode("I can't log in")
b = encode("Password reset failed")
c = encode("Please add dark mode")

print(sum(a * b)) # multiply, add
print(sum(a * c))
```

SIMILARITY, BY HAND

$a \cdot b = 0.62$ both about sign-in
 $a \cdot c = 0.09$ unrelated

every vector has length 1, so
"similarity" is just a dot product

That is the whole search engine: **multiply matching numbers and add them up**. No keywords, no rules, no training.

Search that finds what you meant.

THE CODE

```
Q = "we are being charged wrong"

# keyword: does the text contain it?
tickets[tickets.text.contains(Q)]

# semantic: which vectors align?
sims = E @ encode(Q)
tickets.iloc[sims.argsort()[-3:]]
```

WHAT YOU GET BACK

KEYWORD 0 results

SEMANTIC

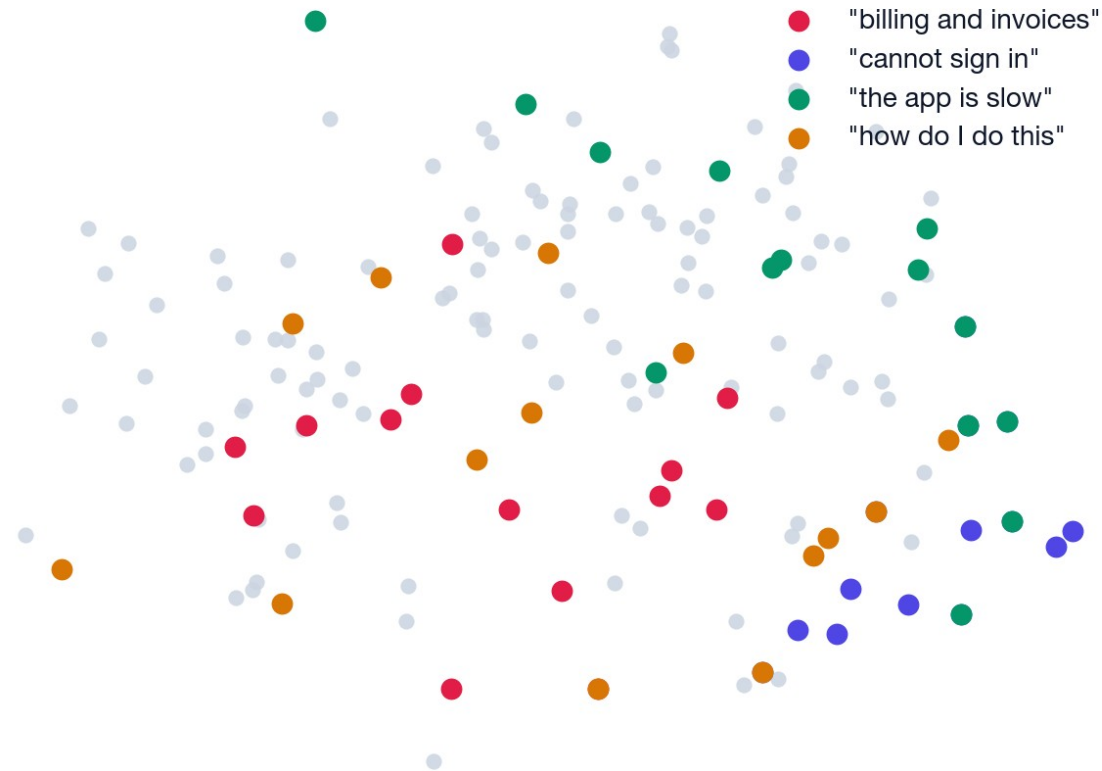
0.51 "We got charged for a trial I understood to be free."
0.35 "invoice INV-88214 doesn't match the amount that hit our card"
0.34 "We were charged twice this month for the same seats."

Not one of those tickets contains the phrase. **Meaning matched, spelling did not.**

Structure was already in the text.

Nobody labelled these. Plot the vectors and the neighbourhoods appear on their own.

160 tickets, arranged by meaning — nobody labelled them



Sort the inbox with zero labelled examples.

DESCRIBE THE BUCKETS IN PLAIN ENGLISH

```
CATEGORIES = {  
  "billing": "an invoice, payment,  
  refund or pricing question",  
  "slow" : "the product being slow,  
  timing out or hanging",  
  ...  
}
```

```
router = pipeline(  
  "zero-shot-classification",  
  model="deberta-v3-zeroshot")
```

```
router(ticket,  
  candidate_labels=CATEGORIES)
```

ROUTED 160 TICKETS

labelled examples used	0
models trained	0
cost	\$0

random guessing	12.5%
top-1 accuracy	64%
top-2 accuracy	80%

You did not train anything. You **described** what you wanted, in English.

It is not a classifier. It is a fact-checker.

The model judges whether a sentence YOU wrote is true of the ticket.

premise: "We were charged twice this month for the same seats."

This example is **an invoice, payment, refund or pricing question.**

ENTAILMENT
+3.29

This example is **the product being slow, timing out or hanging.**

NOT ENTAILED
-3.86

Every category becomes a sentence: "This example is {your description}." The model scores all eight and picks whichever it most believes.

Underneath the one-line pipeline.

WHAT THE PIPELINE DOES UNDERNEATH

```
pair = nli_tok(ticket, hypothesis)
nli(**pair).logits
```

```
# the model's own labels:
# {0: 'entailment',
# 1: 'not_entailment'}
```

IT IS A FACT-CHECKER, NOT A CLASSIFIER

It was never trained to classify tickets.
It was trained to answer one question:

**"given this text, is that
sentence true?"**

Zero-shot just writes one sentence
per category and asks it 8 times.

The words you write ARE the program.

Careful description

"an invoice, payment, refund,
subscription plan or pricing question"

64%

Lazy one-word label

"billing"

47%

Same model. Same tickets. **Only the wording changed: 17 points.** You are not naming a category, you are writing the sentence the model has to fact-check. This is the same skill as writing a good prompt.

An average is a bad way to run a product.

Ask WHICH tickets it gets wrong. The weak buckets are the ones whose descriptions overlap.



When a classifier underperforms, look at your taxonomy before you look at your model.

Confident is not the same as correct.

A TICKET THAT BELONGS IN NONE OF THE BUCKETS

```
result = router(  
    "I saw your careers page and I'd  
    like to apply for the sales role.",  
    candidate_labels=CATEGORIES)  
  
sum(result["scores"])
```

IT PICKED ONE ANYWAY

61% requesting a new capability
9.8% a chart or screen showing
something incorrectly
8.8% connecting to another product

scores sum to **1.00**

Softmax spreads 100% of the belief across the options you supplied. There is no "none of these" unless you build one. Confidence is *"of your choices, which fits best"* , not *"how likely am I to be right"*.

Give the model a way to say "I don't know".

TWO ONE-LINE DEFENCES

```
# fix 1 - somewhere honest to go
CATEGORIES["other"] = (
    "unrelated to the product,
    such as sales or careers")

# fix 2 - score independently
router(ticket, candidate_labels=cats,
multi_label=True)
```

WHAT YOU GET BACK

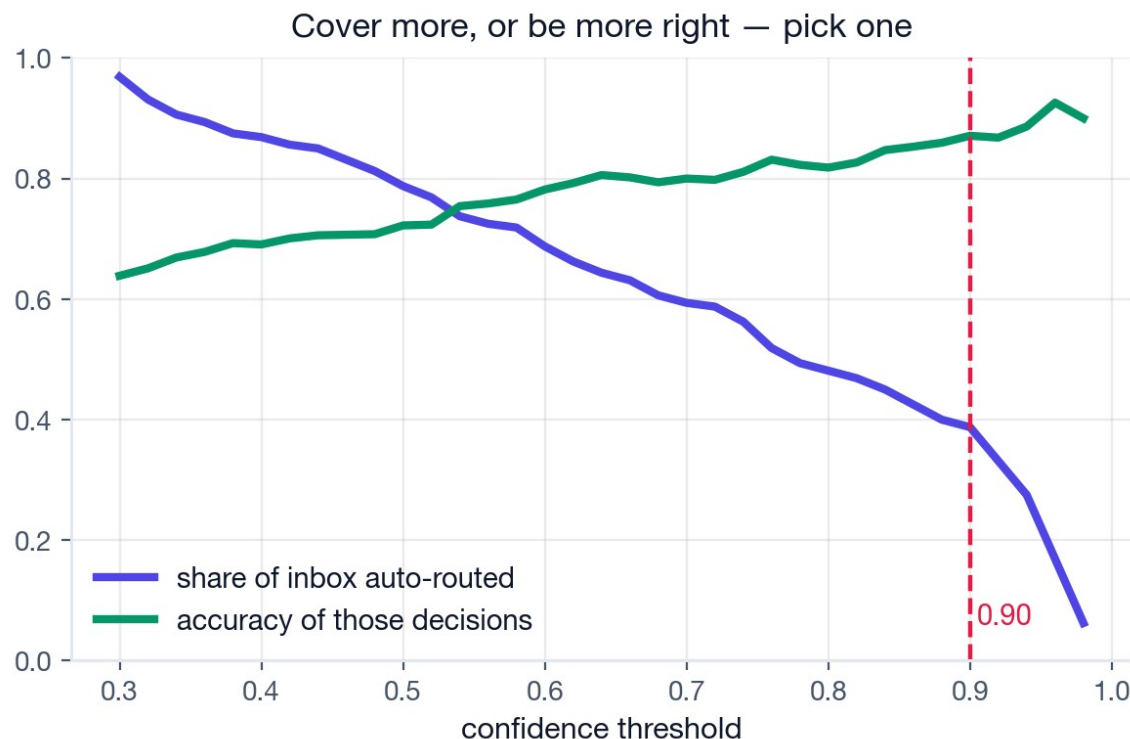
```
with an "other" bucket
  other 75%

scored independently
  13% feature request
  0.3% bug / UI
  sums to 0.14 , low everywhere
  = "I don't know"
```

The difference between a demo and a product is often just this:
deciding what happens when the model is out of its depth.

This curve is the business decision.

There is no setting that maximises both. Where you stand on it is a judgement about cost.



SET IT AT 0.90 AND SHIP

39%

of the inbox handled automatically

87%

accurate on those decisions

~1,937

tickets a week nobody has to touch

"We can automate 39% of the inbox at 87% accuracy, or cover far more and be less right." That is a decision an executive can actually make.

FULL CIRCLE

What we shipped today, for \$0.

No labelled data. No training. No GPU bill. One afternoon.

Semantic search

find any ticket by meaning, across the whole inbox

An 8-way router

built with **zero** labelled examples

A confidence policy

auto-handle 39% at 87% accuracy

An escalation path

the model can now say "I don't know"

Someone else already spent millions teaching a model to read English. Your job is not to build intelligence. It is to point it at your problem, cheaply, and know where it breaks.

WHY THIS MATTERS AT WORK

Six instincts, not six tools.

Every row is a call your future team needs someone in the room able to make.

What you learned today	Why it matters	What it looks like on the job
Tokenization your jargon gets shredded	Semantic search fails on specialist vocabulary	Test the tokenizer before promising 'AI search' on your data
Embeddings distance = similarity	Search, dedupe, clustering, recommendations	Ship 'find similar tickets' without a training set
Zero-shot describe it in English	Useful v1 with no labelled data at all	Ship this week; decide on labelling later
Label wording the words are the program	Precision in writing is now an engineering skill	Same muscle as prompt engineering
Softmax confidence it must pick from your list	A confident answer can be completely wrong	Never gate an irreversible action on a confidence score
Coverage vs accuracy you cannot maximise both	This curve is the actual business decision	Bring the trade-off to the room, not an accuracy number

NEXT TIME

The layer that acts.



What if we DO label?

a few hundred examples, and what they buy you



Where the reading came from

BERT, and how pretraining actually worked



Pull out the specifics

which feature, what error, which plan



Watch it make things up

and why that sets up Week 8

Notebook: L13_Cobalt_Support_Copilot.ipynb. Run it, turn the knobs, break it.

ISBA 2411 · Natural Language Processing & AI · Summer 2026